

Almacenamiento Local con Isar

Guia completa para implementar almacenamiento local embebido con Isar Community en Flutter, siguiendo Clean Architecture.

BASE DE DATOS

**Isar Community ·
NoSQL embebida**

BACKEND

**Supabase (BaaS) ·
Cache local**

NIVEL

Intermedio

TIEMPO ESTIMADO

3-4 horas

Isar

Flutter

Clean Architecture

Cache

Supabase

Contenido

3 capitulos · 3 archivos · ~3-4 horas de lectura

PARTE 1 · FUNDAMENTOS DE ISAR

01 1. Introducción a Isar Community

PARTE 2 · MODELOS Y OPERACIONES

02 2. Modelos y Operaciones con Isar

PARTE 3 · IMPLEMENTACION

03 3. Implementación de Local DataSource



1. Introducción a Isar Community

¿De qué trata esta guía? De entender qué es Isar Community, por qué usarlo, cómo se integra en un proyecto Flutter con Clean Architecture.

Índice

1. ¿Qué es Isar?
2. `isar_community`
3. Instalación y setup
4. Arquitectura de Isar
5. Code generation
6. Checklist

1. ¿Qué es Isar?

Isar es una base de datos embebida NoSQL diseñada específicamente para Flutter y Dart. A diferencia de SQLite (que usa tablas y SQL), Isar almacena objetos Dart directamente como documentos, similar a MongoDB pero sin servidor.

Características principales

Característica	Descripción
Embebida	No necesita servidor. La BD vive en el dispositivo
NoSQL	Almacena objetos Dart directamente, sin conversión SQL
Rápida	Escrita en Rust/C++, hasta 10x más rápida que SQLite en benchmarks
Tipada	Code generation genera APIs completamente tipadas
Isolates	Operaciones asíncronas que no bloquean la UI
Índices	Soportados: único, compuesto, hash, case-sensitive
Multi-plataforma	Android, iOS, macOS, Windows, Linux, Web
Relaciones	<code>IsarLink</code> e <code>IsarObjects</code> para relaciones embebidas o referenciadas

¿Por qué Isar y no otra cosa?

En el contexto de Clean Architecture, Isar se usa como **almacenamiento local** en la capa de Data, específicamente en `LocalDataSource`. Antes de Isar, el estándar era SharedPreferences para datos simples o SQLite (sqflite) para datos estructurados.

Aspecto	SharedPreferences	sqflite	Isar
Tipo de datos	Pares clave-valor (strings)	Tablas SQL relacionales	Documentos NoSQL (objetos Dart)
Tipado	✗ Manual (json.decode)	✗ Manual (Row → objeto)	✓ Automático (code gen)
Velocidad	Rápido (síncrono)	Medio	Rápido (async + isolates)

Aspecto	SharedPreferences	sqflite	Isar
Índices	✗ No	✓ Sí	✓ Sí
Queries complejas	✗ No	✓ SQL	✓ where/filter
Multi-plataforma	✓ Sí	⚠ Limitado	✓ Sí
Ideal para	Configs, tokens simples	Datos relacionales	Cache de datos + sesión

2. isar_community

`isar_community` es el package activo y mantenido del ecosistema Isar, compatible con Dart SDK < 4.0 y la opción recomendada para producción hoy. El package original fue abandonado por su creador en 2023.

Regla simple: usa siempre `isar_community`. Los imports usan `package:isar_community/isar.dart`.

3. Instalación y setup

pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  isar_community: ^3.3.2  
  path_provider: ^2.1.0      # Para obtener el directorio de la app  
  
dev_dependencies:  
  isar_community_generator: ^3.3.2  
  build_runner: ^2.4.0
```

🚀 Inicialización

Isar necesita un directorio donde almacenar los archivos de la base de datos. El patrón recomendado es un **singleton** que se inicializa al arrancar la app:

```

// lib/core/data/local/isar_service.dart
import 'package:isar_community/isar.dart';
import 'package:path_provider/path_provider.dart';
import 'isar_models/cached_user.dart';
import 'isar_models/cached_token.dart';
import 'isar_models/cached_profile.dart';
import 'isar_models/cached_payment_method.dart';

class IsarService {
  static Isar? _instance;

  static Isar get instance {
    if (_instance == null) {
      throw Exception(
        'Isar not initialized. Call IsarService.initialize() first.',
      );
    }
    return _instance!;
  }

  static Future<void> initialize() async {
    if (_instance != null) return;

    final dir = await getApplicationDocumentsDirectory();

    _instance = await Isar.open(
      [
        CachedUserSchema,
        CachedTokenSchema,
        CachedProfileSchema,
        CachedPaymentMethodSchema,
      ],
      directory: dir.path,
      name: 'rifame',
    );
  }

  static Future<void> close() async {
    await _instance?.close();
    _instance = null;
  }

  static Future<void> clear() async {
    await _instance?.writeTxn(() async {
      await _instance!.clear();
    });
  }
}

```

Integración con GetIt (DI)

```
// lib/core/di/injection.dart (fragmento)
import 'package:get_it/get_it.dart';
import 'package:isar_community/isar.dart';

final sl = GetIt.instance;

Future<void> initDependencies() async {
  // Isar
  await IsarService.initialize();
  sl.registerLazySingleton<Isar>(() => IsarService.instance);

  // Local DataSources
  sl.registerLazySingleton<AuthLocalDataSource>(
    () => AuthLocalDataSourceImpl(sl<Isar>()),
  );
  sl.registerLazySingleton<ProfileLocalDataSource>(
    () => ProfileLocalDataSourceImpl(sl<Isar>()),
  );
  sl.registerLazySingleton<PaymentMethodLocalDataSource>(
    () => PaymentMethodLocalDataSourceImpl(sl<Isar>()),
  );

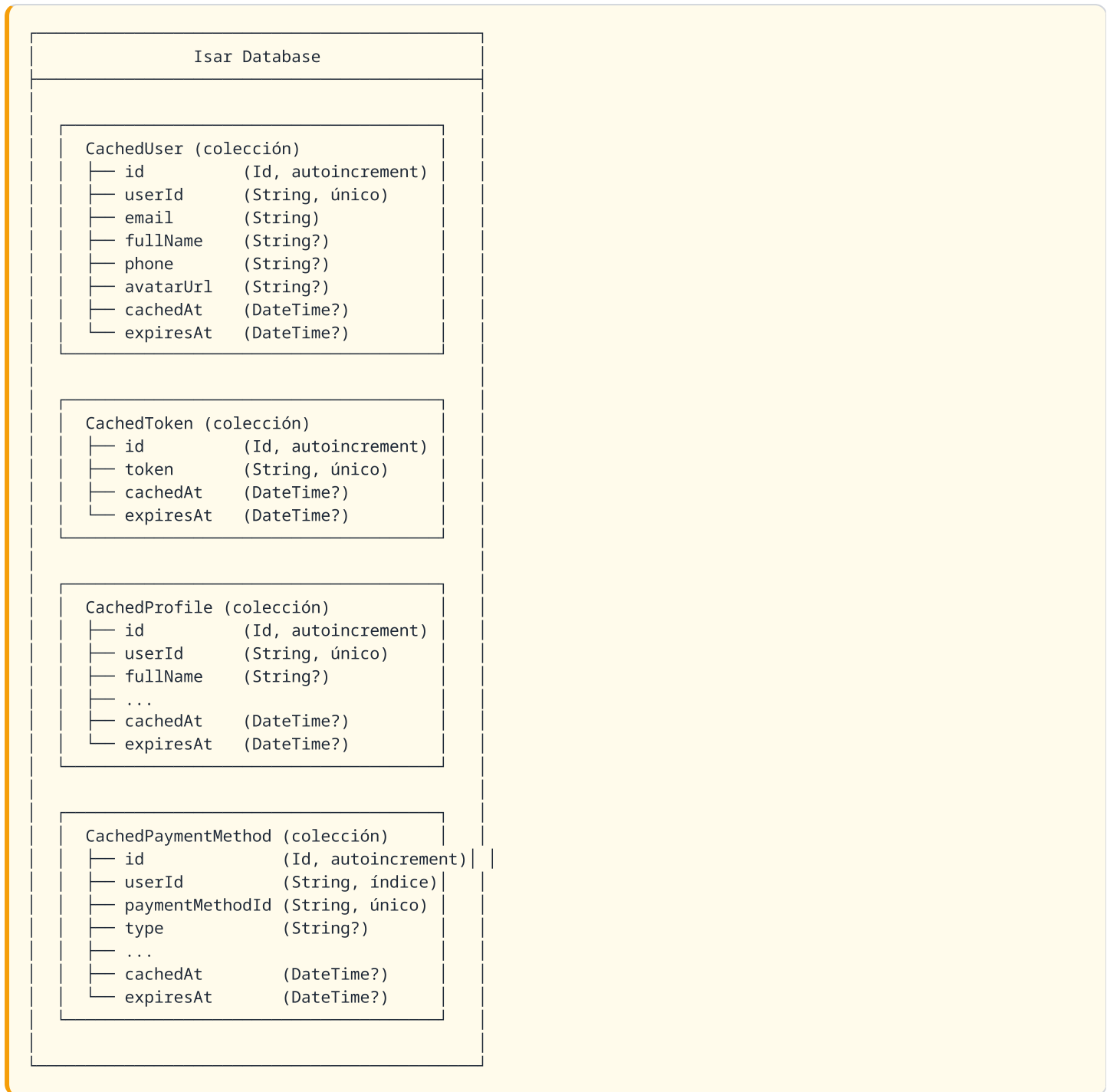
  // Session
  sl.registerLazySingleton<UserSession>(
    () => UserSessionImpl(sl<Isar>()),
  );

  // CacheManager
  sl.registerLazySingleton<CacheManager>(() => CacheManager());
}
```

4. Arquitectura de Isar

Modelo de datos

Isar organiza los datos en **colecciones** (similares a tablas en SQL). Cada colección es una clase Dart anotada con `@Collection()`.



🔑 Conceptos clave

Concepto	Descripción
Colección	Clase Dart con <code>@Collection()</code> . Equivale a una tabla
Esquema	Clase generada automáticamente (<code>NombreSchema</code>). Se pasa a <code>Isar.open()</code>
Id	Campo de tipo <code>Id</code> (int). Puede ser autoincremental o asignado manualmente
Índice	<code>@Index()</code> en un campo. Acelera queries. <code>unique: true</code> para valores únicos
IsarLink	Relación 1:1 o 1:N entre colecciones
IsarObject	Objeto embebido dentro de otro (sin colección propia)

5. Code generation

Cómo funciona

Isar genera código automáticamente a partir de tus anotaciones. Por cada clase con `@Collection()`, se genera:

1. `NombreSchema` — constante con metadatos de la colección
2. Serializador/Deserializador — convierte entre objeto Dart y almacenamiento binario
3. Query builders — `where()`, `filter()`, `sortBy()` tipados
4. Extensiones de acceso — `isar.colecciones` (getter tipado)

Comandos

```
# Una sola ejecución
dart run build_runner build --delete-conflicting-outputs
```

```
# Modo watch (regenera automáticamente al cambiar)
dart run build_runner watch --delete-conflicting-outputs
```

Archivos generados

```
lib/core/data/local/isar_models/
├── cached_user.dart           # ← Tu código
├── cached_user.g.dart         # ← Generado
├── cached_token.dart
├── cached_token.g.dart
├── cached_profile.dart
├── cached_profile.g.dart
├── cached_payment_method.dart
└── cached_payment_method.g.dart # ← Generado
```

Buenas prácticas

- **No editar archivos** `.g.dart`. Se sobrescriben en cada rebuild.
- **Incluir** `*.g.dart` en `.gitignore` o trackearlos (debate abierto). En la práctica, trackearlos evita problemas de versión.
- **Ejecutar** `build_runner` **después de**: agregar/editar colecciones, agregar/editar índices, cambiar tipos de campos.

6. Troubleshooting comun

Error: "Could not find generated Isar schema"

Causa: No ejecutaste `build_runner` despues de crear o modificar una coleccion.

```
dart run build_runner build --delete-conflicting-outputs
```

Error: "Class X is not a registered Type"

Causa: Olvidaste agregar el `Schema` a `Isar.open()`.

```
_instance = await Isar.open(
  [CachedUserSchema, CachedTokenSchema], // <-- Agrega aqui todos los schemas
  directory: dir.path,
);
```


Error: "Conflicting outputs"

Causa: Archivos `.g.dart` corruptos o en conflicto.

```
dart run build_runner build --delete-conflicting-outputs
```

Error: "Isar has already been opened"

Causa: Intentaste abrir Isar dos veces (ej: en tests sin cerrar la instancia anterior).

```
// En tests, usa un nombre unico por test group
final isar = await Isar.open(
  [CachedUserSchema],
  directory: dir.path,
  name: 'test_${UniqueKey().toString()}', // Nombre unico
);
```

Error: "The getter 'X' is not defined for type 'IsarCollection'"

Causa: Los archivos `.g.dart` no estan generados o estan desactualizados.

```
# Limpia y regenera
dart run build_runner clean
dart run build_runner build --delete-conflicting-outputs
```

7. Checklist

- ☐ Usé `isar_community` (fork mantenido activamente)
- ☐ Agregué el enlace a https://pub.dev/packages/isar_community como referencia
- ☐ Agregué `isar_community` y `isar_community_generator` a pubspec.yaml
- ☐ Configuré `IsarService` como singleton con lazy initialization
- ☐ Registré Isar en GetIt (o tu DI container)
- ☐ Ejecuté `build_runner build` exitosamente
- ☐ Verifiqué que los archivos `.g.dart` se generaron
- ☐ Los imports usan `package:isar_community/isar.dart`



Referencias

- [isar_community | pub.dev](#) — Paquete activo de Isar para Dart/Flutter
- [Isar | Documentación original](#) — Referencia de operaciones y queries (legacy)

Nivel: Principiante

Siguiente: 02-modelos-operaciones.md



2. Modelos y Operaciones con Isar

¿De qué trata esta guía? De aprender a definir colecciones Isar con anotaciones, realizar operaciones CRUD dentro de transacciones, construir queries con filtros y ordenamiento, e implementar el patrón TTL (Time-To-Live) para cache expiration.

Índice

1. Definición de colecciones
2. Anotaciones principales
3. CRUD básico
4. Transacciones (writeTxn)
5. Queries con where()
6. Filtros avanzados con filter()
7. Ordenamiento, offset y límite
8. Patrón TTL (Time-To-Live)
9. Casos reales del monorepo
10. Checklist

1. Definición de colecciones

Toda colección Isar es una clase Dart anotada con `@Collection()`. Cada instancia de la clase es un "documento" en la colección.

Estructura de archivos

```
lib/core/data/local/isar_models/  
├── cached_user.dart  
├── cached_user.g.dart  
├── cached_token.dart  
├── cached_token.g.dart  
├── cached_profile.dart  
├── cached_profile.g.dart  
├── cached_payment_method.dart  
└── cached_payment_method.g.dart ← generado
```

Reglas fundamentales

1. Toda colección debe tener un campo `Id id` (puede ser autoincremental o asignado)
2. Los tipos soportados son: `bool`, `int`, `double`, `String`, `DateTime`, `Uint8List`, `List<String>`, objetos anidados con `@embedded`
3. Todos los campos deben ser nullable (`String?`, `DateTime?`) a menos que tengan valor por defecto
4. Se requiere `part 'nombre.g.dart';` al inicio del archivo

2. Anotaciones principales

@Collection()

Marca una clase como colección Isar. Opcionalmente puede especificar `inheritance: true/false` para incluir campos de superclases.

@Index()

Crea un índice en el campo para acelerar queries. Opciones:

Parámetro	Descripción	Ejemplo
<code>unique: true</code>	Valores únicos (como PRIMARY KEY)	<code>@Index(unique: true)</code>
<code>type: IndexType.hash</code>	Índice hash (solo igualdad, más rápido)	<code>@Index(type: IndexType.hash)</code>
<code>composite: [...]</code>	Índice compuesto (varios campos)	<code>@Index(composite: ['campo1', 'campo2'])</code>

@enumerated

Almacena un enum como su índice entero (0, 1, 2...) en lugar de string.

@embedded

Marca una clase como objeto embebido (se almacena dentro del documento padre, no como colección separada).

Id

Tipo especial para la clave primaria. `Isar.autoIncrement` asigna automáticamente.

💡 Regla de oro para @Index

Usar `@Index(unique: true)` en el **business key** (el ID del negocio, no el `Id` interno). Esto permite:

- Buscar por `userId`, `token`, `paymentMethodId` de forma eficiente
- Evitar duplicados a nivel de BD
- Hacer `upsert` implícito con `put()` (si el índice único ya existe, se actualiza)

3. CRUD básico

🔧 Crear / Actualizar (`put`)

```
// put inserta si no existe, actualiza si ya existe (por Id o índice único)
await isar.writeTxn() async {
  final cached = CachedUser()
  ..userId = '550e8400-e29b-41d4-a716-446655440000'
  ..email = 'test@example.com'
  ..fullName = 'Test User'
  ..cachedAt = DateTime.now()
  ..expiresAt = DateTime.now().add(const Duration(days: 30));

  await isar.cachedUsers.put(cached);
};
```

Leer (`get`, `findFirst`, `findAll`)

```
// Por Id interno
final user = await isar.cachedUsers.get(1);

// Primer resultado de la colección
final first = await isar.cachedUsers.where().findFirst();

// Todos los documentos
final all = await isar.cachedUsers.where().findAll();

// Síncrono (solo para where().findFirstSync/ findAllSync)
final syncResult = isar.cachedUsers.where().findFirstSync();
```

Eliminar (`delete`, `deleteAll`)

```
await isar.writeTxn(() async {
  // Por Id interno
  await isar.cachedUsers.delete(1);

  // Todos los documentos
  await isar.cachedUsers.where().deleteAll();

  // Por condición
  await isar.cachedUsers.where().userIdEqualTo(userId).deleteAll();
});
```

4. Transacciones (`writeTxn`)

¿Por qué son necesarias?

Toda operación de escritura (`put`, `delete`, `clear`) **debe** ejecutarse dentro de una transacción. Las lecturas no requieren transacción.

Patrón básico

```
await isar.writeTxn(() async {
  await isar.cachedTokens.where().deleteAll();
  await isar.cachedTokens.put(newToken);
  // Si algo falla aquí, todo se revierte automáticamente
});
```

Ventajas

Ventaja	Descripción
Atomicidad	Si algo falla, todo se revierte
Aislamiento	Las lecturas no ven cambios a medio escribir
Rendimiento	Múltiples operaciones en una sola transacción son más rápidas

⚠ Error común

```
// ✗ Error: put fuera de writeTxn
await isar.cachedUsers.put(user); // Lanza IsarError

// ✔ Correcto
await isar.writeTxn(() async {
  await isar.cachedUsers.put(user);
});
```

5. Operaciones batch con `putAll()`

`putAll()` permite insertar múltiples objetos en una sola transacción, lo cual es significativamente más rápido que insertar uno por uno:

```
// ✗ Lento: N transacciones separadas
for (final user in users) {
  await isar.writeTxn(() => isar.cachedUsers.put(user));
}

// ✔ Rápido: 1 sola transacción
await isar.writeTxn(() => isar.cachedUsers.putAll(users));
```

Impacto real

Método	1,000 objetos	Tiempo aprox.
<code>put()</code> en loop	1,000 txn	~500ms
<code>putAll()</code>	1 txn	~50ms

Sintaxis por tipo de operación batch

```
// Crear/actualizar varios
await isar.writeTxn(() async {
  await isar.cachedUsers.putAll([user1, user2, user3]);
});

// Eliminar varios por ID
await isar.writeTxn(() async {
  await isar.cachedUsers.deleteAll([1, 2, 3]);
});

// Eliminar todos
await isar.writeTxn(() async {
  await isar.cachedUsers.clear();
});
```

6. Streams reactivos con `watch()`

Isar ofrece `watch()` para escuchar cambios en tiempo real en el cache. Esto es útil para reconstruir widgets automáticamente cuando cambian los datos:

watchObject() — Un solo objeto

```
final stream = isar.cachedUsers.watchObject(userId);

stream.listen((cachedUser) {
  if (cachedUser != null) {
    print('Usuario actualizado: ${cachedUser.name}');
  }
});

// Actualizar el usuario → el stream emite automáticamente
await isar.writeTxn() async {
  final user = await isar.cachedUsers.get(userId);
  user!.name = 'Nuevo nombre';
  await isar.cachedUsers.put(user);
};
```

watchLazy() — Toda la colección

```
final stream = isar.cachedUsers.watchLazy();

stream.listen(() {
  print('La colección de usuarios cambió');
  // Reconstruir UI o recalcular estado
});
```

watchObject() con filtro

```
final stream = isar.cachedUsers
  .filter()
  .isActiveEqualTo(true)
  .watchObject(userId);

stream.listen((activeUser) {
  if (activeUser != null) {
    print('Usuario activo actualizado');
  }
});
```

Patrón común: Reconstruir Cubit al escuchar cambios

```
class UserCubit extends Cubit<UserState> {
  final Isar _isar;
  late final StreamSubscription _subscription;

  UserCubit(this._isar) : super(UserInitial()) {
    _subscription = _isar.cachedUsers.watchLazy().listen((_) {
      _loadFromCache();
    });
  }

  Future<void> _loadFromCache() async {
    final users = await _isar.cachedUsers.where().findAll();
    emit(UserLoaded(users));
  }

  @override
  Future<void> close() {
    _subscription.cancel();
    return super.close();
  }
}
```

Nota: `watch()` es ideal para mantener la UI sincronizada con el cache sin tener que llamar manualmente a `load()` después de cada escritura.

7. Queries con `where()`

El método `where()` inicia una query construida con los índices disponibles. Es la forma más rápida de consultar.

Operadores por tipo de índice

Operador	Descripción	Ejemplo
<code>userIdEqualTo(value)</code>	Igualdad exacta	<code>isar.cachedUsers.where().userIdEqualTo(id)</code>
<code>userIdIsNull()</code>	Es null	<code>isar.cachedUsers.where().userIdIsNull()</code>
<code>tokenStartsWith('jwt_')</code>	Empieza con	—
<code>tokenContains('abc')</code>	Contiene	—
<code>cachedAtBetween(a, b)</code>	Rango de fechas	—
<code>cachedAtGreaterThanOrEqual(date)</code>	Mayor que	—
<code>cachedAtLessThan(date)</code>	Menor que	—

Ejemplo real: `PaymentMethod` por `userId`

```
// Buscar métodos de pago de un usuario específico
final methods = await isar.cachedPaymentMethods
  .where()
  .userIdEqualTo('550e8400-e29b-41d4-a716-446655440000')
  .findAll();

// Conteo síncrono (no requiere await)
final count = isar.cachedPaymentMethods
  .where()
  .userIdEqualTo('550e8400-e29b-41d4-a716-446655440000')
  .countSync();
```

⚡ Síncrono vs Asíncrono

Isar ofrece versiones síncronas de algunas operaciones de lectura. Son útiles para getters que deben ser síncronos:

```
// Asíncrono (retorna Future)
final user = await isar.cachedUsers.where().findFirst();

// Síncrono (retorna directamente)
final user = isar.cachedUsers.where().findFirstSync();
final count = isar.cachedUsers.where().countSync();
```

¿Cuándo usar síncrono? En getters síncronos como `hasCachedUser` o `userId` de `UserSessionImpl`, donde no quieres cambiar la firma del método a `async`.

8. Filtros avanzados con `filter()`

Mientras `where()` usa índices, `filter()` permite filtrar por cualquier campo, aunque no tenga índice. Es más lento pero más flexible.

```
// Combinar where + filter
final result = await isar.cachedPaymentMethods
  .where() // Usa índice de userId
  .userIdEqualTo(userId)
  .filter() // Filtro adicional
  .isActiveEqualTo(true)
  .sortByType() // Ordenar
  .findAll();
```

Operadores de filter

Categoría	Operadores
Igualdad	<code>equalTo()</code> , <code>notEqualTo()</code>
Texto	<code>contains()</code> , <code>startsWith()</code> , <code>endsWith()</code>
Nulos	<code>isNull()</code> , <code>isNotNull()</code>
Booleanos	<code>isTrue()</code> , <code>isFalse()</code>
Rangos	<code>between()</code> , <code>greaterThan()</code> , <code>lessThan()</code>
Grupos	<code>and()</code> , <code>or()</code>

9. Ordenamiento, offset y límite

```
final result = await isar.cachedPaymentMethods
  .where()
  .userIdEqualTo(userId)
  .sortByCreatedAt() // Ascendente por defecto
  .thenByType() // Segundo criterio
  .findAll();

// Descendente
final recent = await isar.cachedPaymentMethods
  .where()
  .sortByCreatedAtDesc() // Más reciente primero
  .limit(10)
  .findAll();

// Paginación
final page = await isar.cachedPaymentMethods
  .where()
  .offset(20)
  .limit(10)
  .findAll();
```

10. Patrón TTL (Time-To-Live)

El problema

Cuando guardas datos localmente, no quieres que sean eternos. El perfil del usuario puede cambiar, los tokens pueden expirar, etc. Necesitas una forma de invalidar la cache automáticamente.

La solución

Cada colección incluye dos campos:


```
DateTime? cachedAt;    // Cuándo se guardó
DateTime? expiresAt;   // Cuándo expira
```

🕒 Validación al leer

```
Future<UserModel?> getCachedUser() async {
  try {
    final cached = await isar.cachedUsers.where().findFirst();
    if (cached == null) return null;

    // ✖ Si expiró, limpiar y retornar null
    if (cached.expiresAt != null &&
        cached.expiresAt!.isBefore(DateTime.now())) {
      await clearCache();
      return null;
    }

    // ✅ Válido, convertir a modelo
    return UserModel(
      id: cached.userId ?? '',
      email: cached.email ?? '',
      fullName: cached.fullName,
      phoneNumber: cached.phone,
      avatarUrl: cached.avatarUrl,
      createdAt: cached.createdAt,
      emailConfirmedAt: cached.emailConfirmedAt,
    );
  } catch (e) {
    throw CacheException(message: 'cache_read_error: $e');
  }
}
```

📊 Estrategias de TTL

Estrategia	Cuándo usarla	Tiempo típico
30 días	Datos de perfil que rara vez cambian	<code>Duration(days: 30)</code>
7 días	Datos semi-estáticos (métodos de pago)	<code>Duration(days: 7)</code>
1 día	Datos que cambian a menudo	<code>Duration(hours: 24)</code>
Token session	Mientras el JWT sea válido	Matchear con <code>exp</code> del JWT

💡 Ventajas del TTL sobre "siempre fresco"

1. **Sin llamadas innecesarias** a la API si los datos están en cache y son válidos
2. **Offline-first parcial** — el usuario ve datos aunque no tenga internet (mientras no hayan expirado)
3. **Auto-limpieza** — no acumulas datos obsoletos

11. Casos reales del monorepo

CachedUser

```
@Collection()
class CachedUser {
    Id id = Isar.autoIncrement;

    @Index(unique: true)
    String? userId;

    String? email;
    String? fullName;
    String? phone;
    String? avatarUrl;
    DateTime? createdAt;
    DateTime? emailConfirmedAt;
    DateTime? cachedAt;
    DateTime? expiresAt;
}
```

Uso: Cache del perfil del usuario autenticado. Se guarda al hacer login/signup y se lee para mostrar datos básicos sin llamar a la API.

CachedToken

```
@Collection()
class CachedToken {
    Id id = Isar.autoIncrement;

    @Index(unique: true)
    String? token;

    DateTime? cachedAt;
    DateTime? expiresAt;
}
```

Uso: Cache del JWT. Permite mantener la sesión abierta entre reinicios de la app.

CachedProfile

```
@Collection()
class CachedProfile {
    Id id = Isar.autoIncrement;

    @Index(unique: true)
    String? userId;

    String? fullName;
    String? phoneNumber;
    String? email;
    String? avatarUrl;
    String? preferredLanguage;
    bool? notificationsEnabled;
    DateTime? createdAt;
    DateTime? updatedAt;
    DateTime? cachedAt;
    DateTime? expiresAt;
}
```

Uso: Cache del perfil detallado del perfil de usuario. Se guarda al editar el perfil y se lee para mostrar datos completos.

CachedPaymentMethod

```
@Collection()
class CachedPaymentMethod {
    Id id = Isar.autoIncrement;

    @Index()                // Índice simple (no único – varios métodos por usuario)
    String? userId;

    @Index(unique: true)
    String? paymentMethodId;

    String? type;
    String? name;
    String? bankName;
    String? accountHolder;
    String? accountNumber;
    String? phone;
    String? email;
    String? walletAddress;
    String? qrCodeUrl;
    String? instructions;
    String? cedula;
    String? bankCode;
    String? blockchainNetwork;
    bool? isActive;
    bool? isDefault;
    DateTime? createdAt;
    DateTime? updatedAt;
    DateTime? cachedAt;
    DateTime? expiresAt;
}
```

Uso: Cache de los métodos de pago de un usuario. Notar que `userId` tiene `@Index()` (no único) porque un usuario puede tener varios métodos de pago, mientras que `paymentMethodId` es único.

Resumen de índices

Colección	Business Key	Índice único	Uso
<code>CachedUser</code>	<code>userId</code>	<code>@Index(unique: true)</code>	Solo un usuario por sesión
<code>CachedToken</code>	<code>token</code>	<code>@Index(unique: true)</code>	Solo un token activo
<code>CachedProfile</code>	<code>userId</code>	<code>@Index(unique: true)</code>	Solo un perfil por usuario
<code>CachedPaymentMethod</code>	<code>paymentMethodId</code>	<code>@Index(unique: true)</code>	Múltiples métodos por usuario

12. Checklist

- ☐ Definí las colecciones con `@Collection()`
- ☐ Agregué `@Index(unique: true)` en los business keys
- ☐ Incluí los campos `cachedAt` y `expiresAt` para TTL
- ☐ Ejecuté `build_runner` y los `.g.dart` se generaron
- ☐ Todas las escrituras están dentro de `writeTxn`
- ☐ Uso `where().findFirstSync()` para lecturas síncronas
- ☐ Implementé validación de TTL antes de retornar datos cacheados
- ☐ Uso `filter()` solo cuando el campo no tiene índice



Referencias

- [isar_community](#) | [pub.dev](#) — Paquete activo de Isar para Dart/Flutter
 - [Isar](#) | [Documentación original](#) — Referencia de operaciones y queries (legacy)
-

Nivel: Intermedio

Siguiente: [03-implementacion-local-datasource.md](#)



3. Implementación de Local DataSource

¿De qué trata esta guía? De implementar el patrón LocalDataSource con Isar en Clean Architecture, incluyendo el IsarService singleton, las implementaciones concretas para Auth, Profile y PaymentMethod, el CacheManager como registro centralizado de limpieza, y el UserSessionImpl para acceso síncrono al usuario autenticado.

Índice

1. IsarService: singleton y ciclo de vida
2. Patrón LocalDataSource
3. CacheException: errores de cache
4. AuthLocalDataSourceImpl
5. ProfileLocalDataSourceImpl
6. PaymentMethodLocalDataSourceImpl
7. CacheManager: limpieza centralizada
8. UserSessionImpl: acceso síncrono al usuario
9. Integración con Repository
10. Checklist

1. IsarService: singleton y ciclo de vida



El patrón

IsarService es un **singleton** que gestiona la conexión a Isar. Se inicializa una vez al arrancar la app y se cierra al cerrar la app.

```
// lib/core/data/local/isar_service.dart
import 'package:isar_community/isar.dart';
import 'package:path_provider/path_provider.dart';
import 'isar_models/cached_user.dart';
import 'isar_models/cached_token.dart';
import 'isar_models/cached_profile.dart';
import 'isar_models/cached_payment_method.dart';

class IsarService {
  static Isar? _instance;

  static Isar get instance {
    if (_instance == null) {
      throw Exception(
        'Isar not initialized. Call IsarService.initialize() first.',
      );
    }
    return _instance!;
  }

  static Future<void> initialize() async {
    if (_instance != null) return;

    final dir = await getApplicationDocumentsDirectory();

    _instance = await Isar.open(
      [
        CachedUserSchema,
        CachedTokenSchema,
        CachedProfileSchema,
        CachedPaymentMethodSchema,
      ],
      directory: dir.path,
      name: 'rifame',
    );
  }

  static Future<void> close() async {
    await _instance?.close();
    _instance = null;
  }

  static Future<void> clear() async {
    await _instance?.writeTxn(() async {
      await _instance!.clear();
    });
  }
}
```

Ciclo de vida

```
App start
|
▼
IsarService.initialize() ← se llama en main() antes de runApp()
|
▼
Los LocalDataSources usan IsarService.instance
|
▼
App close → IsarService.close()
```

En tests

En tests no se usa `IsarService`. Se crea una instancia de Isar directamente con `Directory.systemTemp` y un nombre único. Esto se cubre en detalle en [03e-practica-local-datasource-isar.md](#).

2. Patrón LocalDataSource

📐 Estructura

Cada LocalDataSource sigue el mismo patrón:

```
abstract class FeatureLocalDataSource {  
  // Escribir en cache  
  Future<void> cacheData(...);  
  
  // Leer de cache (retorna null si no hay datos o expiró)  
  Future<Model?> getCachedData(...);  
  
  // Limpiar cache  
  Future<void> clearCache();  
}  
  
class FeatureLocalDataSourceImpl implements FeatureLocalDataSource {  
  final Isar _isar;  
  
  FeatureLocalDataSourceImpl(this._isar);  
  
  // ... implementaciones  
}
```

🎯 Características comunes

1. **Constructor recibe Isar directamente** — sin wrapper, sin mock. Isar se inyecta.
2. **Toda escritura va dentro de writeTxn** — si falla, se revierte.
3. **Toda operación se envuelve en try/catch** — los errores se convierten a `CacheException`.
4. **Patrón replace-all** — antes de escribir nuevo data, se borra lo anterior del mismo tipo.
5. **Validación TTL** — al leer, si expiró se limpia y retorna null.

3. CacheException: errores de cache

Los errores de Isar se convierten a `CacheException` para mantener la capa de Data independiente de la implementación concreta:

```
// lib/core/error/exceptions.dart  
class CacheException implements Exception {  
  const CacheException({required this.message});  
  final String message;  
  
  @override  
  String toString() => message;  
}
```

Luego en el Repository, `CacheException` se mapea a `CacheFailure`:

```
// lib/core/error/failures.dart  
class CacheFailure extends Failure {  
  const CacheFailure({super.message = 'Cache failure', super.code});  
}
```

Flujo de error completo

```
Isar.writeTxn lanza excepción
|
▼
catch → CacheException(message: 'cache_write_error: $e')
|
▼
Repository catch → CacheFailure(message: 'Cache failure')
|
▼
Cubit/BLoC → state con error message
```

4. AuthLocalDataSourceImpl

Interfaz

```
abstract class AuthLocalDataSource {
  Future<void> cacheToken(String token);
  String? getCacheToken(); // Sincrónico
  Future<void> cacheUser(UserModel user);
  Future<UserModel?> getCacheUser();
  bool get hasCachedUser; // Getter síncrono
  bool get hasCachedToken; // Getter síncrono
  Future<void> clearCache();
}
```


Implementación

```
class AuthLocalDataSourceImpl implements AuthLocalDataSource {
    final Isar _isar;

    AuthLocalDataSourceImpl(this._isar);

    @override
    Future<void> cacheToken(String token) async {
        try {
            await _isar.writeTxn(() async {
                // Replace-all: borrar token anterior
                await _isar.cachedTokens.where().deleteAll();

                final cached = CachedToken()
                    ..token = token
                    ..cachedAt = DateTime.now()
                    ..expiresAt = DateTime.now().add(const Duration(days: 30));

                await _isar.cachedTokens.put(cached);
            });
        } catch (e) {
            throw CacheException(message: 'cache_write_error: $e');
        }
    }

    @override
    String? getCachedToken() {
        try {
            final cached = _isar.cachedTokens.where().findFirstSync();
            if (cached == null) return null;

            // Validación TTL
            if (cached.expiresAt != null &&
                cached.expiresAt!.isBefore(DateTime.now())) {
                clearCache();
                return null;
            }

            return cached.token;
        } catch (e) {
            throw CacheException(message: 'cache_read_error: $e');
        }
    }

    @override
    Future<void> cacheUser(UserModel user) async {
        try {
            await _isar.writeTxn(() async {
                await _isar.cachedUsers.where().deleteAll();

                final cached = CachedUser()
                    ..userId = user.id
                    ..email = user.email
                    ..fullName = user.fullName
                    ..phone = user.phoneNumber
                    ..avatarUrl = user.avatarUrl
                    ..createdAt = user.createdAt
                    ..emailConfirmedAt = user.emailConfirmedAt
                    ..cachedAt = DateTime.now()
                    ..expiresAt = DateTime.now().add(const Duration(days: 30));

                await _isar.cachedUsers.put(cached);
            });
        } catch (e) {
            throw CacheException(message: 'cache_write_error: $e');
        }
    }
}
```

```

@override
Future<UserModel?> getCacheUser() async {
  try {
    final cached = await _isar.cachedUsers.where().findFirst();
    if (cached == null) return null;

    if (cached.expiresAt != null &&
        cached.expiresAt!.isBefore(DateTime.now())) {
      await clearCache();
      return null;
    }

    return UserModel(
      id: cached.userId ?? '',
      email: cached.email ?? '',
      fullName: cached.fullName,
      phoneNumber: cached.phone,
      avatarUrl: cached.avatarUrl,
      createdAt: cached.createdAt,
      emailConfirmedAt: cached.emailConfirmedAt,
    );
  } catch (e) {
    throw CacheException(message: 'cache_read_error: $e');
  }
}

@override
Future<void> clearCache() async {
  try {
    await _isar.writeTxn(() async {
      await _isar.cachedUsers.where().deleteAll();
      await _isar.cachedTokens.where().deleteAll();
    });
  } catch (e) {
    throw CacheException(message: 'cache_write_error: $e');
  }
}

@override
bool get hasCachedUser {
  try {
    return _isar.cachedUsers.where().countSync() > 0;
  } catch (e) {
    throw CacheException(message: 'cache_read_error: $e');
  }
}

@override
bool get hasCachedToken {
  try {
    return _isar.cachedTokens.where().countSync() > 0;
  } catch (e) {
    throw CacheException(message: 'cache_read_error: $e');
  }
}
}

```

Puntos clave

Método	Tipo	Síncrono?	TTL?	Replace-all?
cacheToken	Escritura	Async	—	✓ Antes de insertar
getCacheToken	Lectura	Síncrono	✓ Si expiró → limpiar y null	
cacheUser	Escritura	Async	—	✓ Antes de insertar

Método	Tipo	Síncrono?	TTL?	Replace-all?
<code>getCachedUser</code>	Lectura	Async	✓ Si expiró → limpiar y null	
<code>hasCachedUser</code>	Getter	Síncrono (countSync)	✗ No	—
<code>hasCachedToken</code>	Getter	Síncrono (countSync)	✗ No	—
<code>clearCache</code>	Escritura	Async	—	Borra ambas colecciones

`getCachedToken()` y los getters `hasCachedUser` / `hasCachedToken` son **síncronos** para poder usarlos en contextos donde no se puede usar `await` (ej: `UserSessionImpl.userId`).

5. ProfileLocalDataSourceImpl

Interfaz

```
abstract class ProfileLocalDataSource {
    Future<void> cacheProfile(UserProfileModel profile);
    Future<UserProfileModel?> getCachedProfile();
    Future<void> clearCache();
}
```

Implementación

```
class ProfileLocalDataSourceImpl implements ProfileLocalDataSource {
    final Isar _isar;

    ProfileLocalDataSourceImpl(this._isar);

    @override
    Future<void> cacheProfile(UserProfileModel profile) async {
        try {
            await _isar.writeTxn() async {
                // Replace-all: borrar perfil anterior
                await _isar.cachedProfiles.where().deleteAll();

                final cached = CachedProfile()
                    ..userId = profile.userId
                    ..fullName = profile.fullName
                    ..phoneNumber = profile.phoneNumber
                    ..email = profile.email
                    ..avatarUrl = profile.avatarUrl
                    ..preferredLanguage = profile.preferredLanguage
                    ..notificationsEnabled = profile.notificationsEnabled
                    ..createdAt = profile.createdAt
                    ..updatedAt = profile.updatedAt
                    ..cachedAt = DateTime.now()
                    ..expiresAt = DateTime.now().add(const Duration(days: 30));

                await _isar.cachedProfiles.put(cached);
            });
        } catch (e) {
            throw CacheException(message: 'cache_write_error: $e');
        }
    }

    @override
    Future<UserProfileModel?> getCachedProfile() async {
        try {
            final cached = await _isar.cachedProfiles.where().findFirst();
            if (cached == null) return null;

            if (cached.expiresAt != null &&
                cached.expiresAt!.isBefore(DateTime.now())) {
                await clearCache();
                return null;
            }

            return UserProfileModel(
                id: cached.userId ?? '',
                userId: cached.userId ?? '',
                fullName: cached.fullName,
                phoneNumber: cached.phoneNumber,
                email: cached.email ?? '',
                avatarUrl: cached.avatarUrl,
                preferredLanguage: cached.preferredLanguage,
                notificationsEnabled: cached.notificationsEnabled,
                createdAt: cached.createdAt,
                updatedAt: cached.updatedAt,
            );
        } catch (e) {
            throw CacheException(message: 'cache_read_error: $e');
        }
    }

    @override
    Future<void> clearCache() async {
        try {
            await _isar.writeTxn() async {
                await _isar.cachedProfiles.where().deleteAll();
            }
        }
    }
}
```

```

    });
} catch (e) {
    throw CacheException(message: 'cache_write_error: $e');
}
}
}

```

6. PaymentMethodLocalDataSourceImpl

Diferencia clave

A diferencia de Auth y Profile (que tienen una sola entrada por usuario), PaymentMethod tiene **múltiples entradas por usuario**. Por eso:

- Usa `userIdEqualTo(userId).deleteAll()` en lugar de `deleteAll()` simple
- La TTL se valida con `cached.first.expiresAt`
- Convierte la lista de `CachedPaymentMethod` a `PaymentMethodModel` con un mapper

Interfaz

```

abstract class PaymentMethodLocalDataSource {
    Future<void> cachePaymentMethods(
        String userId,
        List<PaymentMethodModel> methods,
    );
    Future<List<PaymentMethodModel>?> getCachedPaymentMethods(String userId);
    Future<void> clearCache(String userId);
}

```

Implementación

```
class PaymentMethodLocalDataSourceImpl implements PaymentMethodLocalDataSource {
    final Isar _isar;

    PaymentMethodLocalDataSourceImpl(this._isar);

    @override
    Future<void> cachePaymentMethods(
        String userId,
        List<PaymentMethodModel> methods,
    ) async {
        try {
            await _isar.writeTxn(() async {
                // Borrar solo los métodos de ESTE usuario
                await _isar.cachedPaymentMethods
                    .where()
                    .userIdEqualTo(userId)
                    .deleteAll();

                for (final m in methods) {
                    final cached = CachedPaymentMethod()
                        ..paymentMethodId = m.id
                        ..userId = userId
                        ..type = m.type
                        ..name = m.name
                        ..bankName = m.bankName
                        ..accountHolder = m.accountHolder
                        ..accountNumber = m.accountNumber
                        ..phone = m.phone
                        ..email = m.email
                        ..walletAddress = m.walletAddress
                        ..qrCodeUrl = m.qrCodeUrl
                        ..instructions = m.instructions
                        ..cedula = m.cedula
                        ..bankCode = m.bankCode
                        ..blockchainNetwork = m.blockchainNetwork
                        ..isActive = m.isActive
                        ..isDefault = m.isDefault
                        ..createdAt = m.createdAt
                        ..updatedAt = m.updatedAt
                        ..cachedAt = DateTime.now()
                        ..expiresAt = DateTime.now().add(const Duration(days: 30));

                    await _isar.cachedPaymentMethods.put(cached);
                }
            });
        } catch (e) {
            throw CacheException(message: 'cache_write_error: $e');
        }
    }

    @override
    Future<List<PaymentMethodModel>?> getCachedPaymentMethods(
        String userId,
    ) async {
        try {
            final cached = await _isar.cachedPaymentMethods
                .where()
                .userIdEqualTo(userId)
                .findAll();

            if (cached.isEmpty) return null;

            if (cached.first.expiresAt != null &&
                cached.first.expiresAt!.isBefore(DateTime.now())) {
                await clearCache();
                return null;
            }
        }
    }
}
```

```

    }

    return cached.map(_toModel).toList();
} catch (e) {
    throw CacheException(message: 'cache_read_error: $e');
}
}

@override
Future<void> clearCache(String userId) async {
    try {
        await _isar.writeTxn() async {
            await _isar.cachedPaymentMethods
                .where()
                .userIdEqualTo(userId)
                .deleteAll();
        });
    } catch (e) {
        throw CacheException(message: 'cache_write_error: $e');
    }
}

PaymentMethodModel _toModel(CachedPaymentMethod c) {
    return PaymentMethodModel(
        id: c.paymentMethodId ?? '',
        userId: c.userId ?? '',
        type: c.type ?? '',
        name: c.name ?? '',
        bankName: c.bankName,
        accountHolder: c.accountHolder,
        accountNumber: c.accountNumber,
        phone: c.phone,
        email: c.email,
        walletAddress: c.walletAddress,
        qrCodeUrl: c.qrCodeUrl,
        instructions: c.instructions,
        cedula: c.cedula,
        bankCode: c.bankCode,
        blockchainNetwork: c.blockchainNetwork,
        isActive: c.isActive,
        isDefault: c.isDefault,
        createdAt: c.createdAt,
        updatedAt: c.updatedAt,
    );
}
}

```

7. CacheManager: limpieza centralizada



El problema

Cuando el usuario cierra sesión, es necesario limpiar **todas** las caches locales: tokens, usuarios, perfiles, métodos de pago. Si cada feature limpia su propia cache por separado, es fácil olvidar una.

💡 La solución: Registry pattern

```
// lib/core/services/cache_manager.dart
class CacheManager {
  final List<Future<void> Function()> _clearFns = [];

  void register(Future<void> Function() clearFn) {
    _clearFns.add(clearFn);
  }

  Future<void> clearAll() async {
    await Future.wait(_clearFns.map((fn) => fn()));
  }
}
```

🔧 Integración

```
// En la inyección de dependencias
final cacheManager = CacheManager();

// Cada feature registra su función de limpieza
cacheManager.register(() => sl<AuthLocalDataSource>().clearCache());
cacheManager.register(() => sl<ProfileLocalDataSource>().clearCache());
cacheManager.register(() {
  final userId = sl<AuthLocalDataSource>().getCurrentUserId();
  return sl<PaymentMethodLocalDataSource>().clearCache(userId);
});
```

Nota: `PaymentMethodLocalDataSource.clearCache(userId)` recibe el `userId` porque los métodos de pago son específicos por usuario. Las demás datasources (Auth, Profile) borran todo porque solo hay un usuario en cache.

🔗 Uso al cerrar sesión

```
Future<void> signOut() async {
  await cacheManager.clearAll(); // Limpia TODO de una vez
  await supabase.auth.signOut();
}
```

Ventajas

Ventaja	Descripción
Centralizado	Un solo <code>clearAll()</code> limpia todo
Extensible	Nuevas features solo llaman a <code>register()</code>
Independiente	Cada DataSource expone su <code>clearCache()</code>
Paralelo	<code>Future.wait</code> ejecuta todas las limpiezas en paralelo

8. UserSessionImpl: acceso síncrono al usuario

🧠 El problema

Muchos componentes necesitan saber el ID del usuario autenticado: para filtrar datos, para construir queries, para decidir si mostrar cierta UI. Pedir el token al backend cada vez es ineficiente. Usar `async` en getters simples es incómodo.

💡 La solución: leer desde Isar sincrónicamente

```
// lib/core/session/user_session.dart
abstract class UserSession {
  /// ID del usuario autenticado (síncrono, lee de Isar).
  String? get userId;

  /// Indica si hay un usuario autenticado (síncrono).
  bool get isAuthenticated;
}
```

```
// lib/core/session/user_session_impl.dart
import 'package:isar_community/isar.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:mobile/core/data/local/isar_models/cached_user.dart';
import 'package:mobile/core/session/user_session.dart';

class UserSessionImpl implements UserSession {
  final Isar _isar;
  final SupabaseClient? _supabase;

  UserSessionImpl(this._isar, {SupabaseClient? supabase})
    : _supabase = supabase;

  @override
  String? get userId {
    final cached = _isar.cachedUsers.where().findFirstSync();
    return cached?.userId;
  }

  @override
  bool get isAuthenticated {
    return _isar.cachedUsers.where().countSync() > 0;
  }
}
```

El parámetro `supabase` es opcional. Su propósito es estar disponible para cuando se necesite acceder a `SupabaseClient` desde el `UserSessionImpl`, pero la lectura del `userId` sigue siendo síncrona desde Isar. El `supabase` puede inyectarse para futuras validaciones (ej: verificar que el `userId` de Isar coincida con el de Supabase).

🔑 ¿Por qué funciona?

- `where().findFirstSync()` es una operación **síncrona** en Isar (no bloquea el thread porque Isar usa isolates internamente)
- `CachedUser` se guarda en Isar al hacer login
- `UserSessionImpl` se inyecta como `UserSession` en toda la app

🔗 Uso típico

```
class GetCachedProfile {
  final ProfileLocalDataSource localDataSource;
  final UserSession userSession;

  Future<UserProfileModel?> call() async {
    final userId = userSession.userId;
    if (userId == null) return null;
    return localDataSource.getCachedProfile();
  }
}
```

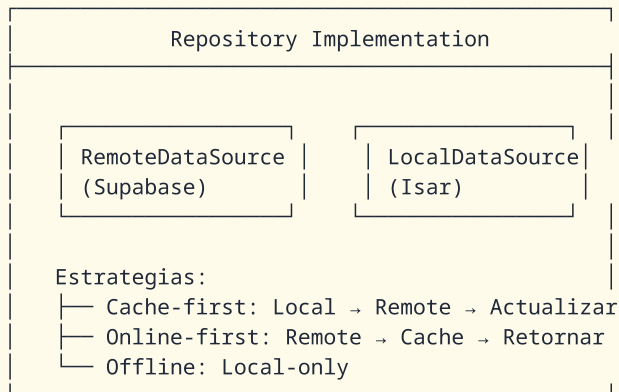
```
class ProfileCubit extends Cubit<ProfileState> {
    final UserSession userSession;

    String get _userId => userSession.userId!; // Sincrónico, sin await
    // ...
}
```

9. Integración con Repository

🎯 Cache-first vs Online-first

En Clean Architecture, el Repository decide la estrategia de cache:



📝 Ejemplo: Online-first para perfil

```
class ProfileRepositoryImpl implements ProfileRepository {
    final ProfileRemoteDataSource remoteDataSource;
    final ProfileLocalDataSource localDataSource;
    final NetworkInfo networkInfo;

    @override
    Future<Either<Failure, UserProfileModel>> getProfile() async {
        try {
            if (await networkInfo.isConnected) {
                final remote = await remoteDataSource.getProfile();
                await localDataSource.cacheProfile(remote);
                return Right(remote);
            } else {
                final local = await localDataSource.getCachedProfile();
                if (local != null) return Right(local);
                return Left(NetworkFailure());
            }
        } on ServerException catch (e) {
            return Left(ServerFailure(message: e.message));
        } on CacheException catch (e) {
            return Left(CacheFailure(message: e.message));
        }
    }
}
```

Ejemplo: Cache-first para auth

```
class AuthRepositoryImpl implements AuthRepository {
  final AuthLocalDataSource localDataSource;

  @override
  bool get isLoggedIn => localDataSource.hasCachedUser; // Síncrono

  @override
  Future<Either<Failure, UserModel>> getCachedUser() async {
    try {
      final user = await localDataSource.getCachedUser();
      if (user != null) return Right(user);
      return Left(CacheFailure(message: 'No cached user'));
    } on CacheException catch (e) {
      return Left(CacheFailure(message: e.message));
    }
  }
}
```

10. Checklist

- ☐ `IsarService` implementado como singleton con `initialize()` / `close()`
- ☐ Todos los `LocalDataSources` reciben `Isar` en el constructor
- ☐ Toda escritura usa `writeTxn`
- ☐ Toda operación envuelta en try/catch → `CacheException`
- ☐ Validación TTL implementada en todas las lecturas
- ☐ `CacheManager` registra la función `clearCache()` de cada feature
- ☐ `UserSessionImpl` implementa `UserSession` con lectura síncrona desde Isar
- ☐ Repository usa `localDataSource.clearCache()` al hacer signOut
- ☐ Los errores `CacheException` se mapean a `CacheFailure` en el Repository

Referencias

- [isar_community | pub.dev](#) — Paquete activo de Isar para Dart/Flutter
- [Isar | Documentación original](#) — Referencia de operaciones y queries (legacy)

Nivel: Avanzado

Siguiente: 03e-practica-local-datasource-isar.md